

DRAFT — System Requirements Specification

Document 3 — Complex Format (High Noise)

This document intentionally includes headers, footers, watermarks, inline annotations, section divider banners, mixed fonts, and a Courier-style body font for selected sections. It is designed to challenge pdfplumber's parsing logic and test the robustness of the extraction script.

FOR TESTING ONLY

PART I — SYSTEM FUNCTIONAL REQUIREMENTS

1 The system shall implement a modular microservices architecture where each service is independently deployable.

[NOTE: The architecture decomposition is defined in the System Architecture Document SAD-001-v3.2. Requirements in this section elaborate the architectural constraints.]

1.1 Each microservice shall expose its API exclusively via the API Gateway; direct service-to-service calls shall not be permitted in production.

1.1.1 The API Gateway shall enforce rate limiting per authenticated client at a default of 1000 requests per minute, configurable per client.

1.1.1.1 Rate limit counters shall be maintained in a distributed cache shared across all API Gateway instances to ensure consistent enforcement.

1.1.1.1.1 The distributed cache shall use a Redis Cluster with a minimum of 3 primary nodes and 1 replica per primary for high availability.

1.1.1.1.1.1 Redis Cluster configuration parameters including node count, replication factor, and timeout values shall be defined in the infrastructure-as-code repository.

1.2 Each microservice shall publish structured logs in JSON format to the centralised logging platform.

1.2.1 Log entries shall include: service name, instance ID, timestamp (ISO 8601 UTC), log level, trace ID, span ID, and message.

1.2.1.1 The trace ID and span ID fields shall conform to the W3C Trace Context specification.

1.2.1.1.1 All services shall propagate the W3C traceparent and tracestate HTTP headers to downstream service calls.

1.2.1.1.1.1 Header propagation shall be implemented using the OpenTelemetry SDK and shall not require manual implementation in individual services.

[ANNOTATION: Trace context propagation is covered by the platform SDK — service teams do not need to implement this independently. Refer to Platform Team runbook PL-TRACE-001.]

1.3 Microservices shall be stateless; session state shall be stored exclusively in the shared session store.

1.3.1 The shared session store shall be backed by Redis with a session TTL of 1800 seconds (30 minutes).

1.3.1.1 Session data shall be encrypted at rest within Redis using the AES-256-GCM algorithm.

PART II — DATA REQUIREMENTS

2	The system shall implement a data model supporting the full lifecycle of engineering requirements from initial capture to archival.
2.1	The data model shall represent requirements as versioned entities where each state change produces an immutable version record.
2.1.1	Version records shall be append-only and shall not be modifiable or deletable by any user role including Administrator.
2.1.1.1	The append-only constraint shall be enforced at the database level using row-level security policies.
2.1.1.1.1	Row-level security policies shall be tested as part of the database migration test suite run before each deployment.
2.1.1.1.1.1	Migration test failures related to row-level security shall block the deployment pipeline and trigger a P1 alert.
<i>[NOTE: Database migration tests are managed by the Platform DB team. Service teams raise a ticket to DB-OPS to include new RLS policies in the test suite.]</i>	
2.2	The system shall store traceability links as first-class entities with their own audit trail independent of the linked artefacts.
2.2.1	Each traceability link entity shall record: source artefact ID and version, target artefact ID and version, link type, creating user, and creation timestamp.
2.2.1.1	When a linked artefact is modified, the system shall create a new link entity referencing the new artefact version and mark the previous link entity as superseded.
2.2.1.1.1	Superseded link entities shall be retained in the database and shall be accessible via the link history view.
2.2.1.1.1.1	The link history view shall display superseded links with a visual indicator distinguishing them from current active links.
2.3	The system shall support soft deletion of requirements artefacts; deleted artefacts shall be marked as deleted and excluded from standard views but retained in the database.
2.3.1	Soft-deleted artefacts shall be restorable by users with the Approver role or higher within 90 days of deletion.
2.3.1.1	After 90 days, soft-deleted artefacts shall be automatically archived to cold storage and shall no longer be restorable through the application interface.

PART III — INTERFACE REQUIREMENTS

3	The system shall provide machine-readable interfaces supporting automated test toolchain integration.
----------	--

3.1	The system shall expose a REST API conforming to OpenAPI Specification 3.1 with a machine-readable schema available at /api/v1/openapi.json.
3.1.1	All API endpoints shall implement HTTP standard methods (GET, POST, PUT, PATCH, DELETE) with semantics conforming to RFC 7231.
3.1.1.1	Mutating operations (POST, PUT, PATCH, DELETE) shall require a CSRF token passed in the X-CSRF-Token request header for browser-based clients.
3.1.1.1.1	The CSRF token shall be a cryptographically random 128-bit value generated per user session.
3.1.1.1.1.1	CSRF token generation shall use the system's cryptographically secure random number generator (CSRNG) and shall not use Math.random() or equivalent.
3.2	The system shall support OAuth 2.0 authorisation code flow with PKCE for API client authentication.
3.2.1	OAuth 2.0 tokens shall have a maximum lifetime of 3600 seconds (1 hour) for access tokens and 86400 seconds (24 hours) for refresh tokens.
3.2.1.1	Refresh token rotation shall be enforced — each use of a refresh token shall invalidate it and issue a new refresh token.
3.2.1.1.1	Refresh token reuse detection shall trigger an immediate invalidation of all tokens in the affected session and an alert to the security operations team.
3.2.1.1.1.1	The reuse detection alert shall include: user ID, token ID, timestamp of original issuance, and timestamp of the reuse attempt.
<i>[ANNOTATION: Refresh token rotation is implemented in the AuthService. API consumers do not need to implement this logic — the SDK handles it transparently. See AUTH-SDK-GUIDE-v2.pdf.]</i>	
3.3	The system shall implement pagination for all list endpoints using cursor-based pagination with a default page size of 50 and a maximum of 500.
3.3.1	Cursor tokens shall be opaque to the client and shall encode the sort key and direction used by the query.
3.3.1.1	Cursor tokens shall have a validity period of 10 minutes to prevent use of stale pagination state.

PART IV — QUALITY AND COMPLIANCE REQUIREMENTS

4	The system shall meet quality and compliance requirements defined by the organisation's quality management system.
4.1	The system shall maintain an uptime of 99.9% measured over any rolling 30-day calendar period.

4.1.1	Uptime shall be measured from the perspective of the external health check endpoint using a third-party monitoring service.
4.1.1.1	Planned maintenance windows of up to 4 hours per month shall be excluded from uptime calculations if communicated 72 hours in advance.
4.1.1.1.1	Maintenance window notices shall be published on the organisation's service status page and sent to all registered users via email.
4.1.1.1.1.1	The service status page shall be hosted independently of the main system so that it remains accessible during system maintenance.
4.2	The system shall retain complete audit logs for a minimum of 7 years in compliance with organisational record retention policy RET-POL-003.
4.2.1	Audit logs shall record all create, read, update, and delete operations on requirement artefacts performed by any user or automated process.
4.2.1.1	Read operations on requirement artefacts with Confidentiality classification of 'Restricted' or above shall be included in the audit log.
4.2.1.1.1	The audit log record for a read operation shall include: user ID, artefact ID, artefact classification, and timestamp.
4.2.1.1.1.1	Audit log records shall not include the full content of the accessed artefact to avoid duplication of controlled content in the audit system.
4.3	The system shall pass an independent security penetration test with no Critical or High findings before each major release.
4.3.1	The penetration test scope shall cover the API, the web application, and the infrastructure layer.
4.3.1.1	Penetration test remediation shall be completed and re-tested before the release is approved for production.

PART V — OPERATIONAL REQUIREMENTS

5	The system shall support operational procedures including deployment, monitoring, incident response, and disaster recovery.
5.1	The system shall support zero-downtime deployments using a rolling update strategy.
5.1.1	The rolling update shall replace no more than 25% of running instances simultaneously.
5.1.1.1	Each replacement instance shall pass the readiness probe before the next batch of instances is updated.
5.1.1.1.1	The readiness probe shall verify database connectivity, cache connectivity, and identity provider connectivity before returning a ready state.

5.1.1.1.1.1	The maximum time allowed for the readiness probe to complete is 10 seconds; probe timeout shall be a deployment configuration parameter.
5.2	The system shall have a documented disaster recovery plan with a Recovery Time Objective (RTO) of 4 hours and Recovery Point Objective (RPO) of 1 hour.
5.2.1	The disaster recovery plan shall be tested with a full failover exercise at least once every 12 months.
5.2.1.1	The results of each DR exercise shall be documented and reviewed by the operations team and the system owner.
5.2.1.1.1	Any failures identified during the DR exercise shall be tracked as defects in the project issue tracker and resolved before the next exercise.
5.2.1.1.1.1	DR exercise defects shall be treated with the same priority as production incidents of equivalent severity.
<i>[NOTE: DR exercise scheduling is coordinated by the Operations team. System owners receive notification 30 days before each scheduled exercise.]</i>	
5.3	The system shall expose metrics in Prometheus exposition format at the /metrics endpoint, accessible only from within the internal monitoring network.
5.3.1	Exposed metrics shall include: request rate, error rate, latency percentiles (p50, p95, p99), and active connection count per service.
5.3.1.1	Custom business metrics shall also be exposed including: active user sessions, requirements created per hour, and baseline creation count.
5.3.1.1.1	Metric names shall follow Prometheus naming conventions with the prefix 'reqmgmt_' for all application-specific metrics.
5.3.1.1.1.1	A complete list of exposed metrics and their descriptions shall be maintained in the operations runbook and updated with each release.

PART VI — NOTIFICATION AND ALERTING REQUIREMENTS

6	The system shall provide a configurable notification and alerting subsystem supporting email, webhook, and in-application channels.
6.1	The system shall allow project administrators to configure notification rules based on requirement state changes.
6.1.1	Notification rules shall support conditions based on attribute values, link events, and baseline creation events.
6.1.1.1	Multiple conditions within a single notification rule shall be evaluated using configurable AND or OR logic.

6.1.1.1.1	Notification rule evaluation shall occur within 30 seconds of the triggering event under normal system load.
6.1.1.1.1.1	If notification delivery fails, the system shall retry delivery using exponential backoff up to a maximum of 3 attempts before marking the notification as failed.
6.2	The system shall provide an in-application notification centre displaying unread notifications to the authenticated user.
6.2.1	Unread notification count shall be displayed as a badge on the notification icon in the application header.
6.2.1.1	The badge count shall update in real time without requiring a page reload, using a WebSocket or server-sent events connection.
<i>[NOTE: Real-time notification delivery is implemented by the NotificationService. See NTF-ARCH-001 for the event bus topology.]</i>	
6.3	The system shall allow users to configure their individual notification preferences, overriding project-level defaults.
6.3.1	User notification preferences shall be stored per project and shall be migrated when a user is transferred between projects.
6.3.1.1	Notification preference migration shall preserve all user-defined settings and shall not revert to project defaults.

PART VII — SEARCH AND DISCOVERY REQUIREMENTS

7	The system shall provide full-text search across all requirement artefacts within a project accessible to the authenticated user.
7.1	Full-text search shall index the Title, Description, and all string-type custom attribute values of each requirement artefact.
7.1.1	The search index shall be updated within 5 seconds of a requirement artefact being created or modified.
7.1.1.1	Search index updates shall be processed asynchronously and shall not block the API response for the triggering write operation.
7.1.1.1.1	If the search index update fails, the failure shall be logged and retried up to 5 times before the artefact is flagged for manual re-indexing.
7.1.1.1.1.1	The manual re-indexing flag shall be visible in the system administration interface alongside the artefact identifier and the failure timestamp.
7.2	The system shall support saved searches that can be named, stored per user, and re-executed on demand.
7.2.1	Saved searches shall support parametric substitution allowing date-range parameters to be updated at execution time without editing the saved query.

7.2.1.1	Parametric substitution shall support relative date expressions including 'today', 'last 7 days', and 'last 30 days'.
7.3	The system shall provide a bulk export of all search results in CSV format containing a configurable subset of requirement attributes.
7.3.1	The bulk export shall be processed asynchronously for result sets exceeding 500 artefacts and delivered to the user via a download link.
7.3.1.1	Download links for async bulk exports shall be valid for 24 hours after generation and shall require authentication to access.
7.3.1.1.1	The system shall notify the requesting user by email when an asynchronous bulk export is ready for download.
7.3.1.1.1.1	The download notification email shall include the export parameters, the record count, and the expiry time of the download link.
7.4	The system shall provide an advanced filter panel supporting multi-attribute filtering on the requirements list view.
7.4.1	The advanced filter panel shall retain the last applied filter set per user per project across browser sessions using persistent local storage.